

FULL ANGULAR NOTES

1. What is Angular?

Angular is a TypeScript-based front-end framework for building single-page applications (SPA).

It is created and maintained by Google.

2. Key Features

- Component-based architecture
- Two-way data binding
- Dependency injection
- Routing system
- RxJS for reactive programming
- TypeScript support

3. Install Angular CLI

```
npm install -g @angular/cli
```

4. Create New Angular Project

```
ng new myapp
```

```
cd myapp
```

```
ng serve -o
```

5. Angular Project Structure

```
src/
```

```
— app/
```

```
— assets/
```

```
— environments/
```

6. Components

Create a component:

```
ng generate component home
```

Files generated:

```
home.component.ts
```

```
home.component.html
```

```
home.component.css
```

```
home.component.spec.ts
```

7. Modules

Every Angular app has at least one module: AppModule.

Create a module:

```
ng g module user
```

8. Templates & Data Binding

Interpolation: `{{ title }}`

Property Binding: `[value]="name"`

Event Binding: `(click)="login()"`

Two-way Binding: `[(ngModel)]="username"`

9. Services

Create service:

```
ng g service auth
```

10. Dependency Injection

Angular can inject services into components automatically.

11. Routing

Example route (escaped to avoid HTML parsing issues):

```
const routes = [  
  { path: 'home', component: HomeComponent },  
  { path: 'login', component: LoginComponent }  
];
```

12. Directives

Structural:

*ngIf

*ngFor

Attribute:

ngClass

ngStyle

13. Pipes

Built-in pipes:

date, uppercase, lowercase, currency

14. Forms

Two types:

- Template-driven
- Reactive Forms

Reactive Form Example:

```
this.form = new FormGroup({  
  username: new FormControl("")  
});
```

15. HttpClient

Import HttpClientModule.

API call:

```
this.http.get('https://api.com/users')
```

16. Observables & RxJS

Example:

```
this.service.getUsers().subscribe(data => console.log(data));
```

17. Lifecycle Hooks

ngOnInit()

ngOnDestroy()

ngAfterViewInit()

18. Angular Build

```
ng build --prod
```

19. Deployment Options

- Firebase Hosting
- GitHub Pages
- Netlify
- Vercel
- Nginx

20. Best Practices

- Use feature modules
- Use services for logic
- Keep components small
- Use interfaces
- Follow a clean folder structure

END OF FULL ANGULAR NOTES